

Day3-01 学习文档 v1.0 : Runtime Architecture —— Agent Runtime 的架构设计原则

本文是《从零实现 Agent Runtime》学习阶段的 Day3-01 正式学习文档。

Day3 被拆分为多个小节。本文是第一节，重点不是立即实现代码，而是建立 Agent Runtime 的架构设计原则：Runtime 应该是 Orchestrator，而不是把 LLM、Tool、Context、Memory、Summary、Streaming 等能力全部堆在一起的万能类。

与上一章的联系

Day2 解决的问题是：

Agent 运行时，到底是谁负责控制流程？

Day2 最终形成了一个关键判断：

LLM 负责推理，Runtime 负责管理。

在这个基础上，Day3 开始进入 Runtime Architecture，也就是回答一个更工程化的问题：

如果现在开始自己写 mini-agent-runtime，第一个版本应该有哪些类？这些类之间是什么关系？

Day3-01 先不着急设计完整类图，而是先建立 Runtime 的架构设计原则。后续 Day3-02、Day3-03、Day3-04 会继续展开核心组件、数据模型与架构图。

目录

Day3 为什么要拆分？

Runtime 不应该是什麼？

Runtime 应该是什麼？

Runtime 与 LLM、Tool 的职责边界

副作用应该属于谁？

为什么 Conversation 不应该知道 Summary？

Dependency Direction：依赖方向

Composition over Inheritance：组合优于继承

纯逻辑与状态边界

Runtime 设计五问

当前阶段的架构判断

下一节学习计划

写书 TODO

写书素材

1. Day3 为什么要拆分？

原始课程规划中，Day3 对应的是：

Chapter3 Runtime Architecture

但在实际学习过程中，发现 Runtime Architecture

不是一个单点知识，而是一个完整的架构问题。它至少包含四个层次：

架构思想
核心组件
数据模型
架构图

因此 Day3 被拆成多个小节：

day03-01-runtime-architecture-design-principles.md
day03-02-runtime-core-components.md
day03-03-conversation-message-data-model.md
day03-04-runtime-architecture-diagram.md

这样做的目的不是增加复杂度，而是避免把不同层次的问题混在一起。

Day3-01 只解决一个问题：

Runtime 为什么应该这样设计？

后续小节再解决：

Runtime 里面到底有哪些对象？

Conversation 和 Message 应该如何建模？

Runtime、ContextBuilder、LLMClient、ToolRegistry 之间如何协作？

2. Runtime 不应该是什？

一个常见错误，是把 Runtime 写成一个不断膨胀的万能类。

例如：

```
class Runtime {  
  conversation  
  llm  
  tools  
  memory  
  summary  
  contextBuilder  
  
  callLLM() {}  
  executeTool() {}  
  buildPrompt() {}  
  saveMemory() {}  
  summarize() {}  
  handleStreaming() {}  
}
```

这个设计一开始看起来很直接，因为所有逻辑都在一个地方。

但随着能力增加，Runtime 很快会变成：

1000 lines
3000 lines
5000 lines

里面会混合：

- Tool 调用

- Memory 管理
- Summary 生成
- LLM SDK 调用
- Prompt 拼接
- Retry
- Workflow
- Streaming
- Event
- Human Approval

这种设计的问题不是“代码多”，而是职责边界消失了。

Runtime 一旦什么都知道，就会出现几个后果：

- ContextBuilder 的逻辑散落在 Runtime 中。
- Tool 的注册、查找和执行逻辑散落在 Runtime 中。
- OpenAI SDK、Claude SDK 等模型供应商细节直接耦合到 Runtime。
- Summary、Memory、Streaming、Human Approval 后续加入时只能继续修改 Runtime。
- 测试时很难替换某一个能力，只能测试整个 Runtime。

所以 Day3 的第一个架构判断是：

Runtime 不应该是能力集合。

3. Runtime 应该是什么？

Runtime 更准确的定位是：

Runtime = Orchestrator

也就是调度器或协调者。

Runtime 自己不应该实现所有业务能力，而应该负责让正确的组件在正确的时机协作。

例如：

Runtime 不应该自己拼 Prompt
Runtime 应该交给 ContextBuilder

Runtime 不应该自己查 Tool
Runtime 应该交给 ToolRegistry

Runtime 不应该直接请求 OpenAI SDK
Runtime 应该依赖 LLMClient Interface

Runtime 不应该自己生成 Summary
Runtime 应该交给 SummaryManager 或 Summary 组件

一个更合理的第一版架构是：

```
Runtime
|
+-- Conversation
+-- ContextBuilder
+-- LLMClient
+-- ToolRegistry
```

Runtime 的职责不是“会做所有事”，而是：

协调所有组件完成 Agent Loop。

如果用类比来理解：

LLM = 负责思考的大脑
Runtime = 负责协调的中枢神经系统
Tool = 负责执行的手、眼、脚
Memory = 长期记忆
Conversation = 经历记录

这个类比需要注意一点：

Runtime 不是负责思考的大脑，真正负责推理和决策生成的是 LLM。

Runtime 更像中枢神经系统：接收 LLM

的决策，检查是否允许执行，调用对应工具，记录结果，再进入下一轮。

4. Runtime 与 LLM、Tool 的职责边界

Day2 已经建立了一个基本判断：

LLM = Decision
Runtime = Orchestration + Control
Tool = Execution

这个判断在 Day3 继续细化。

LLM 负责：

- 根据 Context 生成下一步响应；
- 决定是否建议调用某个 Tool；
- 生成 Tool Call 的 name 和 args；
- 生成最终回答。

Runtime 负责：

- 接收用户输入；
- 写入 Conversation；
- 构建 Context；
- 调用 LLMClient；
- 解析 LLM Response；
- 判断 Tool Call 是否允许执行；
- 通过 ToolRegistry 查找 Tool；
- 执行 Tool；
- 追加 Tool Result；
- 控制循环是否继续；
- 处理超时、最大步数、用户停止和审批。

Tool 负责：

- 执行真实外部动作；

- 返回结构化结果；
- 不直接修改 Runtime 内部状态。

关键点是：

LLM 可以提出动作建议，但 Runtime 拥有执行裁决权。

例如 LLM 返回：

```
{
  "tool": "weather",
  "args": {
    "city": "Shanghai"
  }
}
```

这并不表示 LLM 已经调用了天气工具。它只是请求调用。

真正执行的是 Runtime：

```
const tool = toolRegistry.get("weather")
const result = await tool.execute({ city: "Shanghai" })
conversation.append(result)
```

这条边界是后续 Human Approval、权限控制、Tool Timeout、Retry、审计日志成立的基础。

5. 副作用应该属于谁？

学习过程中有一个重要修正：

副作用不应该简单地全部交给 Tool，而应该交给拥有这个职责的对象。

Tool 的确经常负责副作用，例如：

- 读文件；
- 写文件；
- 发 HTTP 请求；
- 执行 Shell；
- 调用数据库；
- 操作浏览器。

这些副作用属于 Tool，因为 Tool 的职责就是与外部世界交互。

但不是所有副作用都属于 Tool。

例如 Conversation.append(message) 会修改 messages，这也是副作用。但它属于 Conversation 的职责，因为 Conversation 负责维护真实历史。

再例如 SummaryManager 可能会更新 Summary Cache，这也是副作用。但它属于 SummaryManager 的职责，因为 SummaryManager 负责管理摘要状态。

所以更准确的原则是：

副作用属于谁，就放到谁那里。

这背后对应的是：

High Cohesion
Low Coupling

也就是高内聚、低耦合。

一个组件可以有状态，也可以有副作用，但前提是这个状态和副作用必须属于它的职责范围。

6. 为什么 Conversation 不应该知道 Summary ?

Conversation 的职责应该非常窄：

保存真实历史。

它可以提供：

```
conversation.append(message)
conversation.messages
conversation.last()
conversation.clear()
```

但它不应该知道：

- Token 是什么；
- LLM 是什么；
- Summary 是什么；
- Context 是什么；
- Runtime Loop 是什么。

因此，Conversation 不应该出现类似逻辑：

```
if (token > 50000) {
    summary()
}
```

这会让 Conversation 依赖上层策略。

更合理的结构是：

```
Conversation
|
v
SummaryManager
|
v
LLMClient
|
v
SummaryCache
|
v
ContextBuilder
```

Conversation 从头到尾只保存真实消息。

Summary 只是 Runtime 或 SummaryManager 在构建 Context 时使用的辅助状态。

这延续了 Day2 的关键判断：

```
Conversation = Fact
Summary      = Cache
Context      = View
```

7. Dependency Direction : 依赖方向

Day3 补充的第一个架构原则是依赖方向。

依赖应该是单向的：

```
Runtime
|
v
Conversation
```

而不应该是：

```
Conversation
|
v
Runtime
```

原因是 Conversation 是底层事实存储，它不应该知道 Runtime 的存在。

同理：

```
Runtime
|
v
ContextBuilder
```

可以成立。

但不应该让：

```
Conversation
|
v
ContextBuilder
```

因为一旦 Conversation 知道 ContextBuilder，它就会开始知道 Prompt、Token、Summary、Memory 等上层概念。

这会让底层对象被上层策略污染。

因此可以总结为：

底层对象永远不应该知道上层对象。

在 mini-agent-runtime 中，依赖方向应该尽量保持：

```
Runtime
|
+-- Conversation
+-- ContextBuilder
+-- ToolRegistry
+-- LLMClient
```

Runtime 依赖组件。

组件不反向依赖 Runtime。

8. Composition over Inheritance：组合优于继承

Day3 补充的第二个架构原则是：

Agent Runtime 更适合组合，而不是继承。

一个错误方向是：

```
class OpenAIRuntime extends Runtime {}
class ClaudeRuntime extends Runtime {}
class MCPRuntime extends ClaudeRuntime {}
```

```
class HumanApprovalRuntime extends MCPRuntime {}
```

这种继承链看起来可以复用代码，但会很快出现继承爆炸。

因为 Agent Runtime 的变化维度不是一个，而是多个：

能力：Search / Code / Browser / File
策略：Memory / NoMemory / Summary / NoSummary
模型：OpenAI / Claude / Gemini
权限：Auto / Approval / Restricted
流程：Loop / Workflow / Plan-and-Execute

如果用继承表达这些变化，会形成组合数量爆炸。

更合理的方式是组合：

```
const runtime = new Runtime({  
  llm: new ClaudeClient(),  
  memory: new VectorMemory(),  
  tools: new ToolRegistry(),  
  approval: new HumanApproval(),  
  planner: new Planner()  
})
```

这里每个能力都是可替换组件。

Runtime 不需要变成：

```
ClaudeMemoryApprovalPlannerRuntime
```

它只需要组合不同组件。

这和现代前端架构中的 React Hooks 思想类似：

```
function UserPage() {  
  const user = useUser()  
  const permission = usePermission()  
  const data = useFetch()  
}
```

不是通过继承叠加能力，而是通过组合获得能力。

Agent Runtime 也是同一个方向。

9. 纯逻辑与状态边界

学习过程中还讨论了“纯函数组合”的思想。

这个方向是对的，但在 Agent Runtime 中需要稍微修正。

Agent Runtime 不可能完全无状态，因为 Agent 天生需要维护执行过程。

例如：

- Conversation 需要保存 messages；
- Memory 需要保存长期记忆；
- SummaryManager 需要保存 Summary Cache；
- Runtime 需要维护当前 Step、Status、AbortSignal；
- Streaming 需要维护事件序列；
- Human Approval 需要维护等待状态。

所以目标不是：

所有东西都写成纯函数。

而是：

让纯逻辑尽量纯，让状态边界尽量明确，让副作用集中在拥有职责的对象里。

不好的设计：

```
class Runtime {  
  async run() {  
    await openai.chat()  
    await database.save()  
    await tool.execute()  
    await summary()  
  }  
}
```

这里 Runtime 变成了所有副作用的容器。

更合理的设计：

```
class Runtime {  
  async run() {  
    const context = contextBuilder.build(conversation)  
    const response = await llm.chat(context)  
    const result = await toolRegistry.execute(response.toolCall)  
    conversation.append(result)  
  }  
}
```

Runtime 仍然控制流程，但具体能力交给对应组件。

10. Runtime 设计五问

Day3-01 最重要的产出，是形成了一套 Runtime 架构设计检查清单。

以后每增加一个 Agent Runtime 能力，都先回答五个问题。

10.1 Who owns it?

谁拥有这个职责？

例如：

- Prompt 拼接属于 ContextBuilder
- Tool 管理属于 ToolRegistry
- Summary 管理属于 SummaryManager
- 真实历史属于 Conversation
- 模型调用属于 LLMClient

这个问题用于确定职责归属。

10.2 Who decides?

谁拥有决策权？

例如 Tool Call：

- LLM 提出调用建议
- Runtime 决定是否允许执行
- Tool 执行具体动作

再例如 Summary：

- Conversation 不决定是否 Summary
- SummaryManager 不一定决定何时 Summary

Runtime 根据 Token、成本、阶段和策略决定是否触发 Summary

这个问题用于确定控制权。

10.3 Who changes state?

谁负责修改状态？

例如 Conversation：

```
Tool 不应该直接 conversation.messages.push()
LLM 不应该直接修改 Conversation
Runtime 接收结果后调用 conversation.append()
```

状态修改必须有清晰入口。

否则调试、恢复、审计都会变困难。

10.4 Who depends on whom?

依赖方向是什么？

例如：

```
Runtime -> ToolRegistry -> Tool
Runtime -> ContextBuilder
Runtime -> Conversation
Runtime -> LLMClient
```

不应该出现：

```
Tool -> Runtime
Conversation -> Runtime
Conversation -> ContextBuilder
```

这个问题用于避免底层对象依赖上层策略。

10.5 Can this be composed?

这个能力是否应该成为可组合组件？

如果一个能力未来可能：

- 被替换；
- 被扩展；
- 有多个实现；
- 被测试 Mock；
- 被不同 Agent 复用；

那么它就不应该写死在 Runtime 里，而应该设计成组件。

例如：

```
LLMClient
Memory
SummaryManager
Planner
Workflow
ToolRegistry
ApprovalPolicy
```

都应该通过组合接入 Runtime。

11. 当前阶段的架构判断

Day3-01 的核心结论是：

不要设计一个万能 Runtime，而要设计一个负责协调的 Runtime，让能力通过组合不断扩展。

也可以概括为：

```
Core Runtime
+
Pluggable Components
+
Explicit State
+
Controlled Side Effects
```

今天真正建立的不是某个具体 API，而是一套架构判断：

- Runtime 是 Orchestrator，不是所有能力的集合。
- LLM 负责生成决策，Runtime 负责执行裁决。
- Tool 负责外部动作，但不是所有副作用都属于 Tool。
- Conversation 是事实存储，不应该知道 Summary、Context、LLM。
- 依赖方向必须保持单向。
- Runtime 能力应该通过组合扩展，而不是通过继承堆叠。
- Agent Runtime 的难点不是调用 LLM，而是长期运行、可扩展、可恢复、可审计的软件系统设计。

这也是从“会使用 Agent”进入“会设计 Agent Runtime”的关键一步。

12. 下一节学习计划

下一节继续留在 Chapter3 Runtime Architecture 内部：

Day03-02 Runtime Core Components

重点问题是：

Runtime 内部到底为什么需要这些核心组件？

会继续拆解：

```
Runtime
Conversation
Message
ContextBuilder
LLMClient
ToolRegistry
```

Day03-01 解决的是架构原则。

Day03-02 会把这些原则落到第一版核心对象设计上。

13. 写书 TODO

TODO 1：补充「Agent Runtime 设计五问」作为全书核心方法论

建议位置：

Chapter3 Runtime Architecture

|
v
3.1 Runtime Architecture Design Principles

五问包括：

- Who owns it?
- Who decides?
- Who changes state?
- Who depends on whom?
- Can this be composed?

这套问题后续可以贯穿 Memory、MCP、Workflow、Streaming、Human Approval 等章节。

TODO 2：增加一个反例章节：不要设计万能 Runtime

展示错误设计：

```
class Runtime {  
  callLLM() {}  
  executeTool() {}  
  buildPrompt() {}  
  saveMemory() {}  
  summarize() {}  
  handleStreaming() {}  
}
```

说明这种设计会让 Runtime 随着能力增加不断膨胀，最终变成不可维护系统。

然后引出 Orchestrator Pattern。

TODO 3：补充「Agent Runtime 与传统软件架构的关系」

Agent Runtime 不是完全新的软件范式。

它大量继承了：

- 单一职责；
- 高内聚低耦合；
- 面向接口；
- 依赖倒置；
- 组合优于继承；
- 分层架构；
- 状态机；
- 事件驱动。

新的部分在于：

LLM 成为动态决策来源。
Runtime 管理这个动态决策过程。

TODO 4：后续实现 mini-agent-runtime 时验证今天的设计

未来 Chapter12 真正实现时，需要回头验证第一版架构：

```
Runtime  
|  
+-- Conversation
```

+-- ContextBuilder
+-- LLMClient
+-- ToolRegistry

是否能够自然扩展：

- Memory
- MCP
- Workflow
- Human Approval
- Streaming

如果扩展这些能力时 Runtime 需要大量改动，说明当前边界设计仍然不够清晰。

14. 写书素材

素材 1：Runtime 不是大脑，而是中枢神经系统

可以作为 Chapter3 的开篇类比：

LLM = 负责思考的大脑
Runtime = 负责协调的中枢神经系统
Tool = 负责执行的手、眼、脚
Memory = 长期记忆
Conversation = 经历记录

关键观点：

Runtime 不负责产生智能，而负责组织智能完成任务。

素材 2：Agent Runtime 的本质是软件架构问题

很多人认为 Agent 是：

Prompt + LLM + Tool

但生产级 Agent 真正困难的是：

状态管理
生命周期管理
权限控制
错误恢复
事件流转
组件组合
可审计性
可恢复性

因此可以形成一个章节观点：

优秀 Agent Framework 的核心竞争力不是 Prompt，而是 Runtime Architecture。

素材 3：为什么 Agent Framework 更适合组合，而不是继承

反例：

```
BaseAgent
|
v
SearchAgent
|
v
CodingAgent
```

```
|
v
MCPAgent
|
v
HumanApprovalAgent
```

问题是能力维度组合爆炸。

正确方向：

```
new Runtime({
  llm,
  tools,
  memory,
  planner,
  approval
})
```

通过组合获得能力，而不是通过继承扩展 Runtime。

素材 4：Runtime 五问可以作为全书检查清单

例如 Memory 章节可以这样套用：

```
Who owns it?
MemoryManager

Who decides?
Runtime

Who changes state?
MemoryManager

Who depends on whom?
Runtime -> MemoryManager

Can this be composed?
Yes
```

MCP、Workflow、Human Approval、Streaming 都可以用同样结构分析。

素材 5：适合作为 Chapter3 结尾的观点

可以保留以下句子作为书稿素材：

写 Agent Runtime，不是把 Agent 能力堆进 Runtime，而是不断寻找正确的职责边界。

优秀的 Runtime 不是知道所有事情，而是知道应该让谁去做事情。

Agent Runtime 的核心竞争力，不是 Prompt，而是架构。

写 Agent Runtime，本质是在设计一个长期运行、可扩展、可恢复的软件系统。